

JAVA MULTITHREADED CHAT APPLICATION

Project Report

A Terminal-Based Multi-Client Chat Application

Submitted By: Adrutha Leelalakshmi

Reg No.: 25BCE10630

Department: SCOPE

Academic Year:2026-27

Table of Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings & Key Takeaways
13. Future Enhancements
14. References

1. Introduction

The Java Multithreaded Chat Application is a terminal-based, multi-client chat system developed in pure Java. It uses TCP socket communication to allow multiple clients to connect to a central server and exchange messages in real time.

The project demonstrates two important Java concepts: network programming using sockets and concurrent programming using threads and thread-safe collections. A central server accepts incoming connections and creates a dedicated `ClientHandler` thread for each connected client.

The application is intended both as a working chat tool for a local network and as a learning project for understanding client-server communication and multithreading.

2. Problem Statement

A multi-user chat application needs to allow several clients to communicate through a common server at the same time. A server that processes only one client at a time could become blocked while waiting for input, preventing other clients from communicating.

The objective is to design and implement a simple multi-client chat application in which several clients can communicate concurrently, while the server safely manages shared client information and broadcasts messages in real time.

3. Functional Requirements

1. FR1: Start a central chat server on a configurable TCP port.
2. FR2: Accept multiple client connections.
3. FR3: Handle each connected client using a separate server-side thread.
4. FR4: Prompt each client for a username.
5. FR5: Allow clients to send text messages.
6. FR6: Broadcast a client's message to the other connected clients.
7. FR7: Broadcast a notification when a user joins.
8. FR8: Broadcast a notification when a user leaves.
9. FR9: Support /quit for graceful disconnection.
10. FR10: Listen for incoming messages while simultaneously accepting keyboard input.
11. FR11: Allow the server address and port to be configured.

4. Non-functional Requirements

- Performance: Multiple clients should be served concurrently.
- Concurrency: Shared client state must use a thread-safe collection.
- Reliability: One client disconnecting should not stop the server.
- Usability: Terminal prompts and commands should be simple.
- Portability: The application should run on Java SE 8 or later.
- Maintainability: Server, client handling, and client functionality should be separated into clear classes.
- Security limitation: The current version has no authentication or encryption and sends traffic as plain text.

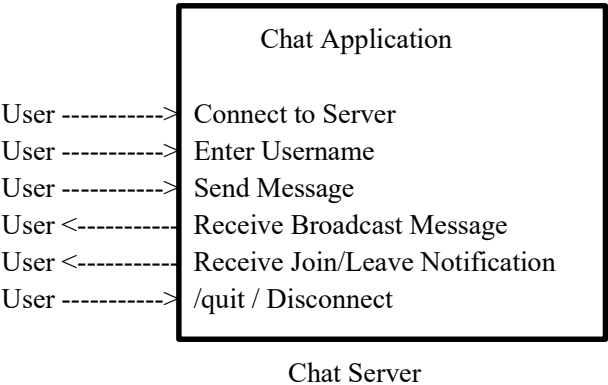
5. System Architecture

The system follows a classic client-server architecture over TCP. ChatServer listens for connections. Each accepted connection creates a ClientHandler thread. The ClientHandler objects share a thread-safe list of active handlers. When a client sends a message, its handler broadcasts the message to the other connected clients.

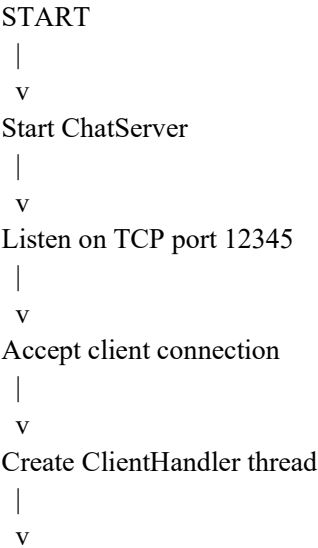
Component	Primary Role	Key Classes & Interfaces
Server Engine	Listens for incoming TCP socket connections on a specific port.	ServerSocket, Socket
Client Handler	Manages continuous read/write cycles for a connected client instance.	Runnable, Thread, BufferedReader
Client Application	Provides interactive user interface for sending & receiving messages.	PrintWriter, InputStreamReader

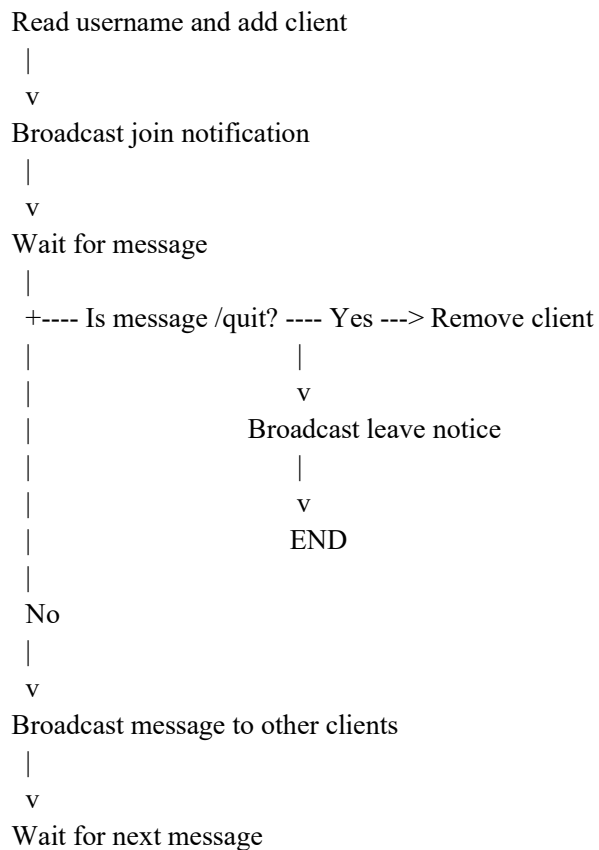
6. Design Diagrams

6.1 Use Case Diagram



6.2 Workflow Diagram





7. Design Decisions & Rationale

- Client-server architecture provides one central point for accepting connections and distributing messages.
- TCP sockets provide reliable and ordered communication.
- The thread-per-client model prevents one client's blocking input from stopping other clients.
- ClientHandler separates per-client communication from server connection management.
- CopyOnWriteArrayList provides safe concurrent access to the active-client collection.
- A background listener thread lets the client receive messages while the main thread waits for keyboard input.
- A simple text protocol keeps the implementation understandable and dependency-free.
- /quit provides a simple explicit graceful-disconnect command.

8. Implementation Details

8.1 ChatServer.java

ChatServer.java is the server entry point. It opens a ServerSocket on port 12345, continuously accepts incoming connections, and creates a new ClientHandler for each connection.

8.2 ClientHandler.java

ClientHandler.java represents one connected client. It reads the username and messages, manages the connection, and broadcasts messages to other connected clients.

8.3 ChatClient.java

ChatClient.java connects to the server, asks for a username, starts a background listener thread for incoming messages, and uses the main thread for keyboard input and sending.

8.4 Communication and Concurrency

A Socket represents one endpoint of a two-way network connection. The server uses ServerSocket to listen on a port; each accepted connection produces a Socket that can be read from and written to.

The thread-per-client model allows multiple clients to be served concurrently. The client also uses a background daemon listener thread because keyboard input and incoming server messages are both blocking operations.

The shared active-client list must be thread-safe. A normal ArrayList is unsafe when multiple threads access it simultaneously. CopyOnWriteArrayList is used to provide safe concurrent access.

8.5 Configuration

ChatServer.java uses PORT = 12345. ChatClient.java uses SERVER_ADDRESS = localhost and SERVER_PORT = 12345. These constants can be changed and the source recompiled for a different port or server address.

9. Screenshots / Results

Server End

```
PS C:\Users\HP> cd C:\Users\HP\OneDrive\Desktop\Chat_App
PS C:\Users\HP\OneDrive\Desktop\Chat_App> java ChatServer
Starting chat server on port 12345...
Server started. Waiting for clients...
New connection from: 127.0.0.1
SERVER: user A has joined the chat!
user A connected.
New connection from: 127.0.0.1
SERVER: user B has joined the chat!
user B connected.
user A: Hello!
user B: Heyy!!
```

Client(s) End

```
PS C:\Users\HP> cd C:\Users\HP\OneDrive\Desktop\Chat_App
PS C:\Users\HP\OneDrive\Desktop\Chat_App> java ChatClient
Connected to chat server. Type /quit to exit.
Enter your username:
user A
SERVER: user A has joined the chat!
SERVER: user B has joined the chat!
Hello!
user A: Hello!
user B: Heyy!!
```

```
PS C:\Users\HP> cd C:\Users\HP\OneDrive\Desktop\Chat_App
PS C:\Users\HP\OneDrive\Desktop\Chat_App> java ChatClient
Connected to chat server. Type /quit to exit.
Enter your username:
user B
SERVER: user B has joined the chat!
user A: Hello!
Heyy!!
user B: Heyy!!
/quit
Disconnected from server.
```

10. Testing Approach

Testing was primarily manual and end-to-end. The implementation was compiled and run with a server process and two separate client processes.

- Start the server and confirm the startup banner appears.
- Connect a single client and verify the connection.
- Send a message and verify handling.
- Connect a second client and verify both clients see each other's messages.

- Use /quit and verify the remaining client sees the leave notification.
- Connect 4–5 clients simultaneously to check concurrent broadcasting.

Reported result: the server correctly handled two concurrent client connections, broadcast join notifications, and relayed chat messages between clients.

11. Challenges Faced

- Handling multiple client connections without blocking the entire server required concurrent execution.
- Shared client information needed protection from unsafe concurrent modification.
- The client needed to receive incoming messages while waiting for keyboard input.
- Graceful disconnection required removing the client and notifying remaining users.
- Moving beyond localhost requires appropriate server-address and port configuration.
- The current design leaves authentication, encryption, persistence, and reconnect behavior for future work.

12. Learnings & Key Takeaways

- Understanding TCP socket communication in Java.
- Practical use of ServerSocket and Socket.
- Understanding the thread-per-client concurrency model.
- Understanding why thread-safe shared state is important.
- Using a background listener thread for simultaneous input and receiving.
- Separating server, client, and connection-handler responsibilities.
- Implementing real-time message broadcasting.

13. Future Enhancements

- Add timestamps to messages.
- Implement private messaging using /msg.
- Add /list to show connected usernames.
- Persist chat history to a file or database.
- Add a Swing or JavaFX graphical interface.
- Use TLS/SSL encrypted sockets.
- Add user authentication and duplicate-username prevention.
- Add reconnect logic after connection drops.

14. References

1. Project technical documentation: Java Multithreaded Chat App, Version 1.0.
2. Java SE standard library documentation for `java.net`, `java.io`, and `java.util.concurrent`.
3. Project source files: `ChatServer.java`, `ClientHandler.java`, and `ChatClient.java`.